



7-etapowy proces **migracji do Kubernetes**

Migracja do Kubernetes – O czym pamiętać, zanim wskoczysz na głęboką wodę

W dynamicznie rozwijającej się branży IT migracja aplikacji do platformy Kubernetes stała się gorącym tematem dyskusji. Dlaczego? Ponieważ decyzja ta nie jest pozbawiona ryzyka i wymaga przemyślanego podejścia. Pomimo że Kubernetes oferuje obiecujące korzyści w zakresie skalowalności i efektywności, nie każdy projekt migracji kończy się sukcesem.

Historie sukcesu często wiodą przewodnią rolę w branżowych dyskusjach, ale równie ważne są te mniej udane, które kończą się frustracją inwestorów oraz zespołów projektowych. Co zatem decyduje o powodzeniu lub niepowodzeniu migracji? Jak można skutecznie przeprowadzić proces decyzyjny i zapewnić, że migracja przyniesie rzeczywiste korzyści, a nie tylko chwilowe zadowolenie z postępu technologicznego?

Inspiracją do tego artykułu stała się aplikacja z sektora finansowego i powiązane z nią wyzwania, gdzie Daniel – jeden ze współautorów – pełnił rolę konsultanta. Przez pryzmat tego przypadku, który przyniósł szereg cennych wniosków, chcielibyśmy przedstawić wnikliwe spostrzeżenia dotyczące procesu decyzyjnego, który rozłożyliśmy na serię klarownych kroków. Zebraliśmy argumenty tej dyskusji w formie procesu konsultacji i podzieliliśmy go na kilka następujących etapów.

- Etap I. Zbieranie informacji
- Etap II. Wybór strategii
- Etap III. Ocena aktualnego rozwiązania klienta i przygotowanego środowiska
- Etap IV. Ułożenie planu ewentualnej migracji
- Etap V. Weryfikacja kosztów infrastruktury
- Etap VI. Projektowanie
- Etap VII. Implementowanie rozwiązania. Migracja

Etap I. Zbieranie informacji



W początkowym etapie pracy nad projektem należy zadać klientowi trzy podstawowe pytania.

1. Skąd decyzja o migracji do Kubernetes?

W diskutowanym przypadku motorem napędowym decyzji było usprawnienie procesu wydawania nowych wersji aplikacji na produkcję. Klient narzekał na poziom skomplikowania i nieprzewidywalność obecnej procedury. Skutkowało to frustracją zespołu technicznego oraz nierzadko nadgodzinami.

Pomijając koszty i stres związany z tego typu incydentami, klient nie chciał być postrzegany jako instytucja niestabilna technicznie. Zaufanie w jego branży jest absolutnie niezbędne a wysoka dostępność rozwiązań IT kluczowa. Technologia Kubernetes jawiła się klientowi jako remedium na te kłopoty, obiecując dodatkowo modernizację dotychczasowego rozwiązania.

Warto nadmienić, że klient – co nietypowe dla sektora finansowego – w pełni wykorzystuje potencjał chmury publicznej Microsoft Azure. Migrację wykonał zgodnie z zaleceniami Cloud Adoption Framework, dlatego jest świadom potencjału, ale także i zagrożeń, jakie niesie ze sobą to środowisko. Posiada znikomą ilość infrastruktury hostowanej w prywatnym centrum obliczeniowym

(on-premise), zaś wykorzystywane komponenty w Azure to przede wszystkim rozwiązania typu Platform as a Service (PaaS) oraz Software as a Service (SaaS) z niewielką ilością Infrastructure as a Service (IaaS) w postaci wirtualnych maszyn.

2. Jak wygląda architektura aplikacji?

W omawianym przypadku aplikacja złożona jest z około 50 serwisów działających mniej lub bardziej niezależnie. Należy nadmienić, że nie jest to z pewnością architektura mikroserwisowa – a na pewno nie jest zgodna z jej kanoniczną definicją. *Odsyłamy tutaj do książek Fundamentals of Software Architecture: An Engineering Approach, A Comprehensive Guide to Patterns, Characteristics, and Best Practices oraz Software Architecture: The Hard Parts: Modern Tradeoff Analysis for Distributed Architectures.*

Z tego powodu, w dalszej części publikacji, celowo nie wykorzystuje się określenia "mikroserwis". Co jednak istotne, cykl życia wszystkich komponentów aplikacji jest w pełni zautomatyzowany przy pomocy procesów CI/CD w Azure DevOps. Wydaje się on być całkiem dojrzały i stabilny. Pozwala na przejście konkretnej wersji serwisu pomiędzy wszystkimi środowiskami tejże platformy: developerskim, przedprodukcyjnym i produkcyjnym – właściwie przy pomocy kilku kliknięć myszy i uruchomieniu stosownych automatyzacji.

Innymi słowy, jest to dość zaawansowany, choć niepozbawiony wad (zwłaszcza kontekście wydajności i wpływu na całą aplikację) proces Software Development Lifecycle (SDLC).

3. Jakie jest uzasadnienie biznesowe tej decyzji?

Klient szuka sposobu na to, aby zespół mógł wydawać nowe wersje aplikacji w godzinach swojej pracy, bez obawy, że aplikacja ta będzie w tym czasie niedostępna dla użytkowników. Do tej pory aktualizacja do nowej wersji odbywała się w godzinach nocnych, gdy ruch generowany przez użytkowników w aplikacji był najmniejszy. Dodatkowo było to spore wydarzenie, które niejednokrotnie szło w parze z dużą dawką stresu.

Klientowi zależy na tym, aby wykorzystać najbardziej zaawansowane wydawania nowych wersji, które pozwolą na zachowanie pełnej dostępności aplikacji, umożliwią wykonanie testów na środowisku produkcyjnym, a także w pełni kontrolowany proces udostępniania nowej wersji wybranym użytkownikom. Klient zastanawia się jednak, czy takie wymagania uzasadniają przejście na Kubernetes i czy migracja wiąże się z jakimiś zagrożeniami.

Jak w przypadku każdej decyzji związanej z architekturą, także i ta musi zostać poddana dogłębnej analizie. Dyskusja nad zasadnością migracji musi brać pod uwagę wady i zalety takiego rozwiązania a jej wynik, pozwolić na ocenę, czy obniżenie ryzyka i czasu wdrożeń może zostać osiągnięte za pomocą technologii Kubernetes.

Etap II. Wybór strategii — weryfikacja celu



Warto zastanowić się, jakie strategie wydawania nowych wersji oferowane przez Kubernetes, mogłyby spełnić wymagania klienta. Istnieje wiele metod opisanych w literaturze takich jak np. Canary Release czy Blue Green Deployment. Pytanie brzmi: czy sprawdzą się w tym konkretnym przypadku i spełnią wymagania klienta?

Strategie wydawania nowych wersji (Release)

Na podstawie zebranych informacji wydaje się, że Blue-Green Release jest najlepszym rozwiązaniem. Wynika to bezpośrednio z potrzeby klienta do testowania aplikacji na środowisku produkcyjnym bez udostępniania jej użytkownikom. Inna strategia – Canary Release – takiej możliwości nie daje. Niejako pośrednią opcją wydaje się być Ring-Based, gdzie jednak kluczem do sukcesu jest bardzo dobra analiza i podział użytkowników na grupy (rings), a także wdrożony zaawansowany system feedback'owy. Jako że klient nie posiada obecnie ani takiej wiedzy, ani też narzędzi, Blue-Green wydaje się być najlepszą i najprostszą strategią.

Wersjonowanie aplikacji

Warto tutaj nadmienić, że pomimo dosyć zaawansowanego procesu CI/CD, serwisy aplikacji nie mają zdefiniowanego (a co za tym idzie wdrożonego) procesu wersjonowania. W rezultacie generuje to kilka istotnych problemów:

- API serwisów nie jest wstecznie kompatybilne
- Utrudniony proces rollback'u
- Brak komunikacji wewnątrz zespołów technicznych o zmianach w API

Na chwilę obecną klient próbuje zaadresować ten problem poprzez wydanie nowej wersji całej aplikacji (wszystkich serwisów) na produkcję. Takie rozwiązanie jest dalekie od idealnego, jednak po głębszej analizie wysiłku wymaganego do wdrożenia poprawnego zarządzania wersją aplikacji, klient zdecydowanie priorytetyzuje migrację do Kubernetes i wykorzystanie nowych metodyk wdrożeniowych.

Należy tutaj wyraźnie zaznaczyć, że to podejście powinno być odzwierciedlone w ryzykach migracji. Brak uporządkowanego zarządzania wersją poszczególnych serwisów utrudni osiągnięcie zakładanych przez klienta pozytywnych rezultatów.

Etap III. Ocena aktualnego rozwiązania klienta i przygotowanego środowiska



Jak już wielokrotnie wspomniano, ryzyko i koszty związane z migracją do Kubernetes mogą być spore, dlatego warto wykonać dogłębną analizę i zweryfikować, czy zamiast przenoszenia aplikacji do kontenerów i Kubernetes, nie lepiej jest przygotować równoległe środowisko (oparte o PaaS lub IaaS), które pozwoli wdrożyć strategię Blue-Green Deployment, minimalizując jednocześnie wady migracji.

Należy pamiętać, że takie rozwiązanie ma również swoje wady. Kluczowym wymaganiem klienta jest możliwość testowania aplikacji na identycznym środowisku jak produkcyjne. To skutkuje potrzebą zbudowania dokładnej repliki produkcji – nierzadko dziesiątek różnych komponentów infrastruktury drastycznie podwyższającą koszty operacyjne.

Bezsporną zaletą technologii kontenerowych jest możliwość zagęszczenia ilości serwisów biznesowych na infrastrukturze. Choć rozwiązania z obszarów takich jak bezpieczeństwo, monitoring czy utrzymanie środowiska wydają się być droższe w przypadku Kubernetes, to bazowa infrastruktura może być tańsza niż w przypadku tradycyjnych rozwiązań PaaS i/lub IaaS.

Możliwe ograniczenia związane z branżą klienta

Dodatkowym i istotnym elementem, który może wpłynąć na decyzję o migracji, jest branża klienta. Należy pamiętać, że sektor finansowy jest obwarowany licznymi regulacjami i wymogami zarówno krajowymi jak i międzynarodowymi. Zasadnym jest, aby przeprowadzane studium wykonalności projektu (feasibility study) ujęło również i ten aspekt.

Z rozmów z klientem wiadomo, że technologia kontenerowa oraz jej pochodne nie były analizowane pod tym kątem. Na szczęście dokonano wnikliwej analizy chmury publicznej, która zakończyła się pozytywnym wynikiem, a w następstwie migracją. To oznacza, że potencjalnie największe problemy ze zgodnością z regulacjami są już rozwiązane, szczególnie jeżeli przyjmujemy, że platforma Kubernetes będzie działać w tej samej chmurze. Nie zwalnia to oczywiście z dokonania dalszej, bardziej szczegółowej analizy problemu.

Warto tutaj pośilkować się zarówno doświadczeniami jak i bezpośrednim wsparciem publicznej chmury. Podczas tej weryfikacji warto pamiętać, że zarówno technologia Kubernetes jak i chmura publiczna oferują szeroki zakres funkcjonalności, które pozwalają na wymuszanie i raportowanie zgodności z regulacjami.

Kultura organizacji

Osobnym, lecz równie kluczowym zagadnieniem jest weryfikacja samej organizacji klienta pod kątem gotowości do pracy z technologią Kubernetes. Warto zwrócić uwagę na dojrzałość procesu DevOps/SRE, wiedzę z zakresu konteneryzacji, a także chęć rozwijania się zespołów technicznych w tym kierunku. Wyniki wstępnej analizy pokazały, że wiedza z zakresu tych technologii raczkuje u naszego klienta, jednak kultura organizacji, a także zastosowane dotychczas rozwiązania takie jak: Infrastructure as Code, procesy CI/CD, wdrożona koncepcja zespołu Platform Engineering – ułatwią potencjalną migrację i adopcję Kubernetes.

Etap IV. Ułożenie planu migracji

Zakładając, że wymagania klienta są wystarczającym powodem do migracji na platformę Kubernetes, należy odpowiedzieć na kilka kolejnych pytań:

- Jak przygotować się do migracji?
- Jak wpłynie ona na organizację i codzienną pracę?
- Czy można ten proces zatrzymać w dowolnym momencie i poddać dodatkowej ewaluacji, a następnie się wycofać?

Sugerowany plan migracji do Kubernetes powinien wyróżniać się następującymi cechami:

- ciągła weryfikacja założeń projektu,
- szeroka walidacja techniczna przy możliwie niskich kosztach i minimalnym wpływie na dotychczasową pracę zespołów technicznych,
- świadome zarządzanie wskazanymi ryzykami.

Takie podejście zapewnia wartość dodaną dla organizacji na każdym etapie projektu a klientowi komfort, jaki daje możliwość podjęcia decyzji o ewentualnym wycofaniu się z inwestycji.

Jak zatem ułożyć plan w zgodzie z powyższym?

Tworzenie obrazów kontenerowych

W pierwszej kolejności warto rozszerzyć istniejące procesy CI/CD, poprzez dodanie kroków odpowiedzialnych za budowanie obrazów kontenerowych dla każdego serwisu.

Takie podejście pozwoli szybko zweryfikować, czy serwisy są kompatybilne z technologią kontenerową, czy też wymagają dodatkowego refactoringu. Dodatkowo minimalizuje się ryzyko dużych nakładów finansowych i zaburzenia obecnego planu prac. Co więcej – nawet jeśli migracja ostatecznie się nie powiedzie, to zespół deweloperski zyska kompetencje z zakresu konteneryzacji – tworzenie obrazów kontenerowych zgodnych z dobrymi praktykami jest fantastycznym doświadczeniem. Kolejnym benefitem adopcji kontenerów jest ułatwienie i przyspieszenie dalszego rozwoju oprogramowania. Wystarczy wspomnieć, że technologie kontenerowe pozwalają na łatwiejsze i bardziej spójne powoływanie do życia środowiska deweloperskiego na lokalnych komputerach programistów.

Skanowanie obrazów, czyli krok w stronę security

Na tym etapie warto także wprowadzić skanowanie obrazów pod kątem Common Vulnerabilities and Exposures (CVEs). To pierwszy krok w kierunku poprawy bezpieczeństwa, dający łatwo osiągalne rezultaty.

Doświadczenie pokazuje, że klient, który priorytetyzuje rozwój oprogramowania nad rozwojem infrastruktury skupia się bardziej na statycznym skanowaniu kodu niż zarządzaniu podstawowym elementem

bezpieczeństwa, jakim jest adresowanie CVE. Jest to szczególnie niebezpieczne w dobie szerokiego wykorzystania paczek i bibliotek open-source.

Dodatkowym benefitem skanerów bezpieczeństwa jest tworzenie Software Bill of Materials (SBOM) – sformalizowany i ustandaryzowany zapis zawierający listę składników i komponentów tworzących oprogramowanie. Każde oprogramowanie potrzebuje zależności, bibliotek, frameworków a sama technologia kontenerowa, dodaje nierzadko paczki powiązane z systemem operacyjnym.

Wprowadzając SBOM, klient będzie w stanie zweryfikować nie tylko to, czy sam kod tworzony przez programistów jest bezpieczny, ale także pozwoli na sprawdzenie, czy wszystkie dodane komponenty są niezbędne do uruchomienia aplikacji oraz, czy są bezpieczne i zgodne z regulacjami. Dodatkowo pozwala na błyskawiczne tworzenie raportów, kluczowych w zarządzaniu bezpieczeństwem (np. poprzez identyfikację obrazów używających konkretną wersję wadliwej biblioteki — vide niedawny przypadek luki w paczce log4j).

Takie podejście jest podstawowym, ale i najprostszym w implementacji krokiem w kierunku zaadoptowania procesów Secure Software Supply Chain (patrz [OWASP SAMM 2.0](#) i [OWASP DSOMM](#)).

Weryfikacja tego, jak działają kontenery

Posiadając wdrożoną automatyzację w zakresie budowy i skanowania kontenerów warto rozważyć kolejny krok — zmiana obecnego sposobu uruchamiania serwisów. Aby tego dokonać, należy przeprowadzić dwutorową analizę.

Pierwszy etap wymaga oceny czy obecnie używane komponenty infrastruktury pozwalają na zmianę sposobu hostowania serwisów.

Obecnie jest to osiągnięte poprzez wdrożenie wykonywalnej wersji kodu (np. jar czy dll), jednak wiele platform dostępnych w chmurze Microsoft Azure – jak chociażby Application Services – daje możliwość wdrożenia obrazu kontenerowego zamiast wykonywalnej wersji kodu. To pozwala na ocenę (stosunkowo niskim kosztem) czy zbudowane obrazy pracują wydajnie i przede wszystkim – czy spełniają zadane wymagania funkcjonalne i нефункционалне. Takie podejście pozwala uniknąć skoku na głęboką wodę, jaką bez wątpienia jest budowa i konfiguracja klastra a w następstwie migracja.

Równolegle drugim etapem jest lepsze poznanie architektury aplikacji, zwłaszcza pod kątem zarządzania stanem i skalowalności.

Taka weryfikacja pozwoli na wskazanie tych obszarów całego oprogramowania, które migrować będzie łatwiej, a które trudniej. Jakie komponenty wykorzystają w pełni platformę Kubernetes, a jakie będą wymagały dodatkowego nakładu pracy (np. serwisy zarządzające stanem, wykonujące zadania w tle itp.).

Podsumowując, warto zacząć przygodę z konteneryzacją od rozszerzenia istniejących procesów CI/CD tak, aby produkowały obrazy kontenerowe. Te zaś należy skanować pod kątem CVEs i budowy SBOM. Mając obrazy w repozytoriach, można planować zmianę hostingu na obecnych platformach – z kodu na kontenery.

Etap. V. Weryfikacja kosztów infrastruktury i całego projektu



Wszystkie dotychczas omówione działania nie pociągają za sobą wysokich wydatków. Jednak wcześniej czy później konieczne będzie przeprowadzenie szczegółowej weryfikacji kosztów infrastruktury oraz sporządzenie kosztorysu całkowitego projektu.

Należy sprawdzić, czy wymagania klienta da się odzwierciedlić w postaci listy konkretnych etapów i zadań do wykonania. Warto również przemyśleć, które elementy rzeczywiście powinny znaleźć się w kalkulacji, albowiem pojęcie 'koszty infrastruktury' wymaga precyzyjnego zdefiniowania.

Elementarne koszty klastra Kubernetes

Istnieje wiele możliwości tworzenia, konfigurowania i zarządzania klastrem (zarówno w chmurze publicznej jak i prywatnej). Mając na uwadze, obecność klienta w Azure, naturalnym wyborem zdaje się być Azure Kubernetes Service (AKS). Zarządzana, prekonfigurowana z wbudowanym szeregiem automatyzacji platforma kontenerowa. Jej wstępna wycena, przy pomocy dostępnego kalkulatora, nie nastręcza trudności, zmusza jednak do podjęcia podstawowych decyzji architektonicznych – takich jak: moc obliczeniowa, rozmiar pamięci, konfiguracja sieci, aspektów bezpieczeństwa. To zdecydowanie główny punkt całego kosztorysu, ale niekoniecznie najdroższy.

Koszty związane z monitoringiem



Rozważając migrację do środowiska Kubernetes, należy być świadomym, że może to pociągnąć za sobą zmiany w komponentach odpowiedzialnych za monitorowanie aplikacji biznesowych.

W omawianych przypadku klient monitoruje szczegółowo aplikację głównie za pomocą natywnych mechanizmów Azure. Nasuwa się pytanie: czy te mechanizmy będą nadal efektywne po migracji? To kolejny, kluczowy element analizy w całym procesie migracyjnym.

Monitoring i bezpieczeństwo to dwa najważniejsze aspekty dla naszego klienta, zwłaszcza w kontekście bankowości internetowej, gdzie bezpieczeństwo jest priorytetem. Klienci oczekują niezawodnego dostępu do usług, takich jak sprawdzanie salda konta czy realizacja przelewów. W przypadku jakichkolwiek problemów z aplikacją, bank musi działać proaktywnie, aby wykryć i rozwiązać wszelkie nieprawidłowości, zanim zauważy je klient.

Bezpieczeństwo rozwiązania

Z naszego doświadczenia wynika, że jednymi z najdroższych komponentów kosztorysu są te związane z bezpieczeństwem.

Chociaż Azure oferuje solidne rozwiązania dotyczące bezpieczeństwa klastra Kubernetes – pod warunkiem, że są one odpowiednio zaprojektowane – na rynku dostępne są bardziej zaawansowane opcje. Warto tutaj kierować się proponowanym przez społeczność specjalistów Kubernetes frameworkiem 4C: Code, Container, Cluster, Cloud. Holistyczne podejście do tych czterech obszarów pozwala ocenić poziom dojrzałości praktyk bezpieczeństwa w przedsiębiorstwie. Wybór właściwego stosu narzędzi związanych z security jest procesem długotrwałym, skomplikowanym i wymagającym dogłębnej wiedzy z zakresu bezpieczeństwa we wszystkich powyższych obszarach.

Warto jednak zainwestować czas w taką analizę, gdyż koszty tej klasy narzędzi są znaczące a pokusa oparcia całego rozwiązania o narzędzia open-source wielka i niestety (z naszego doświadczenia) często zwodnicza.

Koszty związane z integracją całego klastra w obecnym środowisku

Migracja do Kubernetes oznacza wprowadzenie nowej warstwy abstrakcji w infrastrukturze, co wymaga specjalistycznych automatyzacji. Dotyczy to między innymi instalacji dodatkowych narzędzi, które są niezbędne do pełnego wykorzystania potencjału klastra. Te działania są nieodłącznym elementem

kosztorysu i powinny być dokładnie przemyślane oraz zaplanowane. Tylko wtedy klaster będzie integralną częścią całej infrastruktury klienta.

Koszty związane ze wsparciem

Koszty wsparcia technicznego stanowią istotny, lecz często niedoceniany element w procesie migracji systemów do środowiska Kubernetes. Istotne jest przeszkolenie obecnej kadry inżynierskiej w zakresie obsługi i administracji Kubernetes, co może wiązać się z koniecznością organizacji szkoleń wewnętrznych lub zewnętrznych kursów certyfikacyjnych.

Ponadto, ze względu na specyfikę i złożoność Kubernetes, często pojawia się potrzeba rozbudowy zespołu wsparcia o specjalistów z doświadczeniem w tej technologii. Nowi pracownicy mogą być niezbędni do zapewnienia ciągłości działania i efektywnego rozwiązywania problemów, które mogą pojawić się w nowym środowisku. Dlatego też, planując migrację, należy uwzględnić potencjalne koszty rekrutacji i utrzymania większej liczby pracowników.

Warto również zauważyć, że odpowiednio przeszkolony zespół jest kluczowy dla optymalizacji kosztów operacyjnych w dłuższej perspektywie. Inwestycja w edukację i rozwój kompetencji pracowników może przynieść znaczące oszczędności poprzez zwiększenie efektywności pracy i minimalizację ryzyka błędów, które mogą prowadzić do kosztownych awarii systemu.

Dodatkowe koszty

Choć pod tą dosyć ogólną kategorią można zmieścić praktycznie wszystko, warto tutaj skupić się na tzw. kosztach ukrytych i często pomijanych. Do tych bezsprzecznie należą podwyższone koszty środowisk testowych, a także developerskich. Przykładem mogą być dodatkowe licencje na Docker Desktop czy dodatkowe narzędzia ułatwiające pracę z kontenerami i wszelkie koszty związane z ich utrzymaniem.

Podsumowanie tematu kosztów

Podsumowując, migracja do Kubernetes może być zaskakująco kosztowna, mimo że jest to technologia często postrzegana jako open-source. Klienci zwykle doświadczają szoku, gdy odkrywają, że "darmowe" rozwiązanie wiąże się nierzadko z wysokimi wydatkami.

Jest to kluczowy moment, aby ponownie ocenić z klientem jego gotowość do migracji i zastanowić się, czy nadal jest to dla niego opłacalne.

Dla architekta ważne jest, aby umożliwić klientowi podejmowanie świadomych decyzji na każdym etapie projektu, dostarczając wartość przy minimalnych kosztach, co wspiera proces decyzyjny i pomaga w osiągnięciu optymalnych wyników.

Etap VI. Projektowanie



Po szczegółowym przedstawieniu klientowi wszystkich kosztów związanych z infrastrukturą Kubernetes, w tym wydatków na dodatkowe narzędzia, kosztów ukrytych, a także omówieniu ryzyk migracyjnych, klient znajdzie się w pozycji, aby podjąć świadomą i przemyślaną decyzję o przejściu na tę technologię.

Zakładając, że klient potwierdzi swoje zainteresowanie i zdecyduje się na "tak" dla migracji, wówczas nasuwa się pytanie, co dalej?

Pierwszym krokiem nie będzie natychmiastowe tworzenie infrastruktury. Zamiast tego, kluczowe jest strategiczne planowanie i przygotowanie gruntownego planu migracji, który uwzględni wszystkie aspekty techniczne i biznesowe projektu. Na tym etapie priorytetem jest stworzenie szczegółowej dokumentacji, która będzie odzwierciedlać podstawowe założenie projektu. Taki krok jest w zgodzie z podstawowym założeniem całego projektu, jakim jest postępowanie małymi krokami, przy możliwie niskich kosztach, ale jednocześnie dostarczanie jasno określonej wartości dla klienta.

Filary dobrej dokumentacji

W początkowej fazie wyceny projektu poziom zrozumienia problemu jest zazwyczaj ogólny i nie ma możliwości zagłębienia się w szczegóły implementacyjne. Szacowana jest złożoność problemów i czas potrzebny na ich rozwiązanie, ale szczegółowa analiza jest zarezerwowana dla etapu projektowania. To właśnie wtedy tworzona jest dokumentacja, która będzie rozumiała zarówno dla biznesu, czyli klienta, jak i dla specjalistów technicznych. Zrozumienie, że nie jest to pojedynczy dokument, jest kluczowe.

Dokument High Level Design (HLD), przeznaczony głównie dla klienta, będzie uporządkowanym zbiorem informacji przedstawionych podczas wyceny. Ciekawostką jest, że zazwyczaj nie ma potrzeby tworzyć go od razu na początku projektu. Łatwiej jest opracować HLD, mając już Low Level Design (LLD) – dokument, który z inżynierskiego punktu widzenia szczegółowo opisuje architekturę rozwiązania i kluczowe aspekty implementacyjne.



Oba dokumenty, HLD i LLD mogą być oparte na pięciu filarach, które proponujemy. Dzięki nim można upewnić się, że wszystkie istotne kwestie związane z projektowaniem infrastruktury i procesami jej utrzymania są kompleksowo rozpatrzone i odpowiednio adresowane.

Oto wspomniane pięć filarów:

1. **Secure** – Zapewnienie bezpieczeństwa całego rozwiązania jest priorytetem. Obejmuje to ochronę przed zagrożeniami zewnętrznymi i wewnętrznymi, a także zabezpieczenie danych i transakcji.
2. **Monitor** – Monitorowanie systemu pozwala na ciągłe śledzenie jego stanu i wydajności, co jest kluczowe dla szybkiego wykrywania i reagowania na wszelkie nieprawidłowości.
3. **Configure** – Dokładne konfigurowanie systemu, w tym decyzje dotyczące warstwy sieciowej, środowiska uruchomieniowego kontenerów, przechowywania danych i innych elementów infrastruktury Kubernetes.

4. **Protect** – Zabezpieczenie ciągłości działania biznesu (business continuity) i planowanie odzyskiwania po awariach (disaster recovery) to kwestie, które mają zasadnicze znaczenie dla trwałości i odporności systemu.
5. **Govern** – Weryfikacja wymagań i zapewnienie zgodności z obowiązującymi regulacjami prawnymi i standardami branżowymi to fundamenty zarządzania i kontroli projektu.

Proces tworzenia dokumentacji High Level Design (HLD) i Low Level Design (LLD) jest nieodłącznym elementem rozwoju projektów IT, a jego czasochłonność jest uzależniona od specyfiki danego przedsięwzięcia. Może to być kwestia dni, tygodni lub nawet miesięcy.

Niezależnie od preferencji, etap projektowania jest etapem obowiązkowym, nieuniknionym, szczególnie w obszarze, gdzie poziom wiedzy na temat technologii kontenerowych w społeczności IT – zarówno w Polsce, jak i na świecie – jest jeszcze stosunkowo niski. Wiele osób wciąż uczy się podstaw, a brak solidnego fundamentu w postaci przemyślanego projektu może prowadzić do katastrofalnych wyników, gdzie finalnie wdrożone rozwiązanie okaże się niefunkcjonalne.

Ostatni, ale równie ważny argument aby traktować priorytetowo etap projektowy i dokumentacyjny, wynika z faktu, że prace nad systemem IT są często przekazywane. Inżynierowie zmieniają projekty, zespoły ewoluują, a bez jasnej i kompleksowej dokumentacji, przekazanie wiedzy byłoby jak nawigacja w gęstej mgle – pełne niepewności i ryzyka błędów. Dlatego właśnie HLD, LLD i zbiór udokumentowanych architektonicznych decyzji (ADR) są nie tylko zapisem technicznym, ale również mapą, która prowadzi przez złożoność projektu i umożliwia jego bezpieczne i efektywne utrzymanie w przyszłości.

Etap VII. Implementowanie rozwiązania. Migracja



Realizacja projektu migracyjnego i płynne przejście przez kolejne fazy wymaga czasu, który należy mierzyć miesiącami, a nawet kwartałami. Zmiany, jakie niesie ze sobą migracja, są znaczące i wymagają starannego planowania oraz wykonania – nic nie dzieje się automatycznie.

Kwestia czasu poświęconego na realizację projektu jest istotnym elementem dyskusji z klientem. To kolejna decyzja, która powinna być podjęta po dokładnej analizie wszystkich za i przeciw. Należy wspólnie rozważyć, czy korzyści wynikające z migracji przewyższają nakład czasu i zasobów, co jest kluczowe dla ustalenia, czy przedsięwzięcie to jest nadal opłacalne i strategicznie uzasadnione.

Istnieje wiele sposobów i metodyk migracji aplikacji do klastra Kubernetes. Warto tutaj wspomnieć przynajmniej o dwóch z nich.

Podejście service by service

Podejście service by service do migracji serwisów do środowiska Kubernetes jest strategią, która pozwala na stopniowe wdrażanie zmian przez wszystkie etapy, aż do środowiska produkcyjnego. Jest to metoda, która umożliwia szybką weryfikację sensowności całego procesu migracyjnego. Wykorzystanie tej metody to przedsięwzięcie o wysokim stopniu skomplikowania, które opiera się na zasadzie "wszystko albo nic". Sukces tego podejścia mierzy się zdolnością do wprowadzenia na produkcję co najmniej jednego komponentu.

Strategia ta, niesie ze sobą ryzyko związane z równoczesnym funkcjonowaniem dwóch środowisk infrastrukturalnych: nowego, opartego na Kubernetes, oraz starego, tradycyjnego modelu. Przykładowo, migracja 5 z 50 komponentów do Kubernetes stawia przed nami wyzwanie utrzymania ich na zadanym poziomie dostępności (np. 99,9%). W przypadku wystąpienia problemów pojawia się dylemat lokalizacji ich źródła – czy leży ono w nowym środowisku Kubernetes, czy też w dotychczasowej infrastrukturze. Dodatkowo istotne jest, aby zespół deweloperski posiadał odpowiednie kompetencje do

diagnozowania i rozwiązywania problemów, które mogą różnić się w zależności od platformy PaaS w chmurze czy Kubernetes.

Mimo wad migracja service by service posiada również znaczące zalety. Przede wszystkim, daje ona możliwość obserwacji działania nowego rozwiązania w praktyce, co może stanowić kolejny punkt decyzyjny. Jeśli na podstawie kilku serwisów udowodnimy, że migracja end to end nie jest tak problematyczna, jak mogłoby się wydawać, może to stanowić silną motywację do kontynuacji procesu migracji w tym stylu.

Podejście environment by environment

Alternatywną strategią migracji jest metoda "środowisko po środowisku", która zakłada stopniowe przenoszenie poszczególnych środowisk do klastra Kubernetes, począwszy od środowiska developerskiego. Taka sekwencyjna migracja umożliwia kompleksowe testowanie systemu w nowym środowisku kontenerowym, zamiast ograniczania się do pojedynczych komponentów.

W ramach tego procesu, ruch użytkowników jest przekierowywany do nowego środowiska po zakończeniu prac nad każdym kolejnym etapem. Przykładowo, po pomyślnej migracji środowiska testowego, cały ruch związany z tym etapem jest przekierowywany z dotychczasowych rozwiązań PaaS do Kubernetes. To podejście jest postrzegane jako mniej ryzykowne, ponieważ pozwala na dokładne przetestowanie zmian w środowisku nieprodukcyjnym, co zwiększa bezpieczeństwo całego przedsięwzięcia. Minusem takiego rozwiązania jest stosunkowo późna weryfikacja działania aplikacji na produkcji — rzeczywisty wdrożenie jest odsuwane w czasie do bardzo późnej fazy projektu. Procesy automatyzujące CI/CD stają się na czas trwania inicjatywy bardzo skomplikowane — w ramach każdego serwisu muszą obsługiwać dwa, odrębne stosy technologiczne.

Proces migracji

Przy założeniu, że infrastruktura Kubernetes jest już przygotowana (wliczając w to komponenty takie Ingress Controller, systemy uwierzytelniania i autoryzacji dostępu), można przystąpić do migracji właściwego systemu.

Wykonanie poprzedniego kroku, jakim było skonteneryzowanie serwisów aplikacyjnych, a także przystosowanie automatyzacji CI/CD jest znaczącym ułatwieniem. Następnym, polecanym krokiem jest efektywne wykorzystanie tych obrazów, na przykład poprzez opracowanie manifestów YAML lub Helm Chartów.



Dodatkowym obszarem wartym uwagi na tym etapie są testy. Zespół projektowy powinien być złożony nie tylko z ekspertów Kubernetes, ale również z Test Managera i specjalistów od automatyzacji testów infrastruktury. Dzięki nim zminimalizowana zostanie materializacja ryzyk związanych z migracją, obniżone koszty związane z utrzymaniem infrastruktury i przede wszystkim – ułatwione będzie przekazanie projektu do fazy wsparcia.

Podsumowując, proces migracji można przedstawić w dużym uproszczeniu jako:

1. Wybór strategii.
2. Konteneryzacja poszczególnych elementów systemu (jeśli to konieczne).
3. Opracowanie manifestów YAML lub Helm Chartów.
4. Aktualizacja procesów CI/CD, aby wdrażały manifesty lub Helm Charty zamiast tradycyjnych artefaktów aplikacyjnych.
5. Przeprowadzenie testów we współpracy z architektami i zespołem testerskim.
6. Adaptacja modelu wsparcia do nowego środowiska i przygotowanie na potencjalne incydenty.

Warto migrować do Kubernetes?



Migracja do Kubernetes, choć może wydawać się procesem złożonym i wymagającym znacznych nakładów finansowych, nie powinna być przedmiotem obaw. Wręcz przeciwnie, stanowi ona znaczącą szansę na rozwój i modernizację projektów oraz organizacji. Umożliwia eksplorację nowych horyzontów technologicznych, takich jak DAPR czy Service Mesh, otwierając drzwi do innowacji.

Technologie i narzędzia skoncentrowane wokół ekosystemu Kubernetes mogą wydawać się skomplikowane nawet dla doświadczonych inżynierów, a tym bardziej dla klientów i partnerów biznesowych. Trudność ta wynika z bogactwa dostępnych technologii i narzędzi, które stale ewoluują i ujawniają coraz nowsze detale implementacyjne. Przykładem może być technologia eBPF, która zyskuje na popularności w świecie Kubernetes i generuje wokół siebie wiele nowych narzędzi. Nie wszystkie z nich są łatwe w adopcji na szeroką skalę, co podkreśla potrzebę wsparcia ze strony inżynierów i architektów.

Naszym zadaniem jest przeprowadzenie klienta przez te złożone narzędzia w sposób możliwie najmniej ryzykowny.

Przed przystąpieniem do implementacji, konieczne jest przeprowadzenie dogłębnych dyskusji z klientem, aby wspomóc go w podjęciu świadomej decyzji. Warto zwrócić uwagę klientowi, że Kubernetes nie jest kolejną platformą PaaS, która będzie utrzymywana przez dostawcę chmury bez ich udziału. Zrozumienie, że migracja do tej technologii wiąże się z istotną zmianą w sposobie utrzymania infrastruktury, jest absolutnie kluczowe do osiągnięcia sukcesu.

W przypadku analizowanego klienta z sektora bankowego, jedną z kluczowych zalet migracji jest prowadzenie projektu w sposób nowoczesny, bez obciążenia długiem technologicznym. Projekt spełnia wszystkie wstępne wymagania, co czyni go idealnym kandydatem do migracji.

Migracja do Kubernetes – co może pójść nie tak?

Migracja do Kubernetes jest procesem wymagającym, który może wydawać się przytłaczający ze względu na swoją złożoność i potencjalne koszty. Nie jest to zadanie, które można zrealizować w krótkim czasie; realistyczne oszacowanie zakłada wymiar czasu liczony w kwartałach, a nie dniach czy tygodniach.



Potencjalne wyzwania migracyjne obejmują:

1. **Ocena przydatności konteneryzacji:** Istotne jest przeprowadzenie dogłębnej analizy, czy aplikacja nadaje się do konteneryzacji, jeśli jeszcze nie została skonteneryzowana. W przypadku już skonteneryzowanych aplikacji, kluczowe jest zrozumienie głównych powodów migracji i porównanie ich z alternatywnymi rozwiązaniami, takimi jak natywne platformy chmurowe.
2. **Ocena gotowości firmy:** Należy zweryfikować, czy firma jest gotowa do adopcji technologii Kubernetes. Być może rozwiązania prostsze takie jak Docker Compose czy Docker Swarm będą łatwiejsze i bardziej uzasadnione.

Przykładem może być środowisko, w którym zespół wsparcia technicznego edytuje pliki YAML za pomocą narzędzi z interfejsem graficznym, takich jak Portainer. W przypadku Docker Compose, prostota plików YAML pozwala na łatwe wprowadzanie zmian bez głębokiej wiedzy technicznej. Natomiast w przypadku Kubernetes, złożoność plików konfiguracyjnych wymaga od osób odpowiedzialnych za wdrożenie znaczenie głębszego zrozumienia technologii. Dlatego migracja do Kubernetes jest planowana tylko wtedy, gdy zespoły inżynierów i architektów będą miały czas na opracowanie odpowiednich procesów CI/CD, umożliwiających automatyzację i eliminujących potrzebę ręcznej edycji plików konfiguracyjnych.



Dodatkowe aspekty do rozważenia:

- **Oszacowanie kosztów i korzyści:** Każda decyzja migracyjna powinna być uzasadniona ekonomicznie, czy to w perspektywie krótko, czy długoterminowej. Jeśli serwisy są używane przez ograniczoną liczbę osób, migracja może nie być zasadna.
- **Zrozumienie procesu migracji:** Migracja wymaga dostosowania wielu aspektów operacyjnych, takich jak odświeżanie certyfikatów SSL, zapewnienie wysokiej dostępności, zarządzanie poufnością danych, integracja z systemami CI/CD, oraz zarządzanie trwałym zapisem danych. Wszystkie te elementy muszą być wzięte pod uwagę przy podejmowaniu decyzji.

Podsumowując, migracja do Kubernetes wymaga starannego planowania, zrozumienia technologii i procesów, a także oceny gotowości organizacji do podjęcia tego kroku. Jest to zadanie, które może przynieść znaczące korzyści, ale tylko wtedy, gdy jest przeprowadzone w sposób przemyślany i z odpowiednią wiedzą.

Skąd wziąć więcej informacji o migracji do Kubernetes?

Poszukiwanie informacji na temat migracji do Kubernetes jest kluczowym krokiem w przygotowaniu się do tego procesu. Internet oferuje bogate źródło materiałów edukacyjnych, które mogą posłużyć jako solidna podstawa do zrozumienia i wdrożenia tej technologii.



Warto zacząć od zgłębienia działania tej technologii, najlepiej poprzez praktyczne przykłady zbliżone do warunków produkcyjnych. W tym kontekście, szkolenie [Kubernetes Maestro](#) oraz jego aktywna społeczność, licząca ponad 1300 absolwentów, może okazać się nieocenionym źródłem informacji i wsparcia. Szkolenie to nie tylko przygotowuje do certyfikacji CKA i CKAD, ale również umożliwia wymianę doświadczeń z innymi specjalistami.

Dla tych, którzy rozważają migrację do chmury Microsoft Azure, zalecane jest zapoznanie się z materiałami dostarczanymi przez Microsoft, w tym Well-Architected Framework oraz Azure Architecture Center. Te zasoby oferują przegląd najlepszych praktyk i sprawdzonych architektur, w tym te wykorzystujące klastry Kubernetes, i mogą służyć jako przewodnik po złożonościach architektury w chmurze.

Ważnym aspektem jest również bezpieczeństwo kontenerów, gdzie wspomniany framework 4C dostarcza solidnych wytycznych. Jest on szczegółowo opisany w dokumentacji Kubernetes i stanowi niezbędne źródło wiedzy dla każdego, kto chce zrozumieć kluczowe aspekty bezpieczeństwa w kontekście konteneryzacji.

Nie można pominąć strategii wydawania nowych wersji, takich jak Blue Green i Canary Release, które są dobrze udokumentowane i stanowią ważny element procesu ciągłego dostarczania oprogramowania.

Na koniec, warto podkreślić znaczenie kultury i mindsetu DevOps w organizacji, co jest niezbędnym warunkiem wstępnym do migracji do Kubernetes. Książka [Continuous Delivery](#) autorstwa Jez Humble oraz [Szkolenie Automation Maestro](#) mogą być doskonałymi źródłami wiedzy na temat wprowadzania kultury DevOps i nowoczesnych modeli dostarczania oprogramowania.

Autorzy



Bartłomiej Czudek - Architekt IT, specjalizujący się w Microsoft Azure i Kubernetes



Damian Naprawa - Docker Captain, Cloud & Kubernetes Architekt, Założyciel wkontenerach.pl



Daniel Musiał - Architekt IT, specjalizujący się w Microsoft Azure i Kubernetes

Bibliografia

1. [Fundamentals of Software Architecture: An Engineering Approach. A Comprehensive Guide to Patterns, Characteristics, and Best Practices](#)
2. [Software Architecture: The Hard Parts: Modern Tradeoff Analysis for Distributed Architectures](#)
3. [Azure Kubernetes Service](#)
4. [Szkolenie Kubernetes Maestro](#)
5. [OWASP SAMM](#)
6. [OWASP Devsecops Maturity Model](#)
7. [Framework 4C: Cloud, Cluster, Container, Code](#)
8. [Azure Architecture Center](#)
9. [Szkolenie Automation Maestro](#)
10. [Continuous Delivery: Reliable Software Releases Through Build, Test, and Deployment Automation](#)